EFES Master Node: From Sound Waves to Persistent Storage, a System on Chip Walkthrough

Gioele Giunta
s360501

Politecnico di Torino
LM 32, Computer Engineering
Course: Electronics for Embedded Systems

Academic Year 2025 / 2026

Abstract

This report documents the master node of the EFES project, an acoustic communication system between two ESP32 based devices where one of the two is paired with a Tang Nano 20K FPGA running a custom System on Chip. The focus is on the FPGA side: the SoC architecture, the on chip bus, the hardware accelerator pipeline that goes from the SPI microphone into block RAM, through a streaming FFT, then into SDRAM where a tone decoder consumes it, the PWM speaker output, and the external W25Q64FV NOR flash that holds persistent logs and settings via a finite state machine driven by the FPGA itself. The document walks top down: first the system as a whole, then the SoC block diagram with its bus map, then each internal module in detail with its state machine, then the external connections (NOR flash, SDRAM, op amp, speaker), then a chapter on the signal integrity problems we had to fight and the way we solved them in firmware and in hardware. A final chapter covers the ESP32 firmware that talks to the FPGA using register level UART access. Where useful the report includes timing diagrams and zoomed views. The closing chapter summarizes what worked, what was tried and abandoned, and what could be improved with a proper PCB instead of jumper wires.

Contents

1 Introduction **4**
- 1.1 What this report focuses on . . . 4
- 1.2 Reading map . . . 4

2 System overview **5**
- 2.1 Bird eye view . . . 5
- 2.2 The master, opened up . . . 5

3 The FPGA SoC, top level view **6**
- 3.1 High level block diagram . . . 7
- 3.2 Bus and address map . . . 7
- 3.3 What changed from the course template . . . 8

4 Component deep dives **8**
4.1 The soft CPU . . . 8
4.2 Clock tree (rPLL plus per peripheral prescalers) . . . 9
4.3 SDRAM controller . . . 10
4.3.1 Init sequence . . . 10
4.3.2 Problems we had . . . 11
4.4 Hardware accelerator pipeline . . . 11
4.4.1 The FFT engine . . . 11
4.4.2 Audio decoder, parabolic peak interpolation . . . 12
4.4.3 The window radius problem . . . 12
4.5 PWM speaker . . . 12
4.6 NOR flash controller (`flash_ctrl`) . . . 12
4.6.1 Memory layout . . . 13
4.6.2 Top level state machine . . . 13
4.6.3 The SCAN strategy . . . 14
4.6.4 Block protection unlock at boot . . . 14
4.6.5 Polling tolerance . . . 15
4.7 UART char modules . . . 15
4.8 Timer and GPIO . . . 15

5 Outside the FPGA **15**
5.1 The W25Q64FV NOR flash chip . . . 15
5.1.1 Inside the NOR cell . . . 16
5.1.2 Wiring . . . 16
5.2 Embedded SDRAM die . . . 16
5.3 SPI MEMS microphone . . . 17
5.4 Op amp speaker buffer . . . 17
5.5 ESP32 connections . . . 17

6 Signal integrity, voltage droops, decoupling **18**
6.1 The W25Q64FV brownout during sector erase . . . 18
6.2 First capacitor: 1 µF . . . 18
6.3 Second capacitor: 10 µF in parallel . . . 18
6.4 SPI clock speed reduction . . . 19
6.5 Retry plus verify on the ESP32 side . . . 19
6.6 The fallback FFL diagnostics . . . 19

7 ESP32 firmware **20**
7.1 Architecture . . . 20
7.2 Register level UART, instead of HardwareSerial . . . 20
7.3 flash_link with retry and verify . . . 21
7.4 Defensive read of settings . . . 21

8 The acoustic protocol **21**
8.1 Codebook . . . 21
8.2 CSMA on the air . . . 22
8.3 Retries . . . 22

9 What worked, what did not **22**
9.1 Things that worked end to end . . . 22
9.2 Things we tried and discarded . . . 23
9.3 What still wobbles . . . 23
9.4 Numbers from the bench . . . 23

10 Conclusions **23**

A Quick reference card **24**

1 Introduction

The EFES project ("Electronics for Embedded Systems") is the final deliverable for the course of the same name at Politecnico di Torino. The brief asks for a simple acoustic communication system between two embedded nodes: a master and a slave. The two boards talk to each other through the air, using audible tones, with the master also acting as a Wi Fi gateway toward a remote dashboard. The system has to demonstrate the use of a custom SoC on FPGA, a microcontroller for higher level logic, external memory, and persistent storage.

This document is the report for the master node. The master is the more complex of the two devices because it carries:

- an ESP32 dev board acting as the application processor and Wi Fi radio (also exposing a USB serial console for development),
- a Tang Nano 20K board carrying a Gowin GW2AR 18 FPGA on which we built the SoC described in this report,
- an external W25Q64FV serial NOR flash chip on a DollaTek breakout module, talking SPI to the FPGA,
- an external SPI MEMS microphone for the audio input path,
- a small PWM speaker output stage with an op amp buffer and an analog low pass filter,
- assorted jumper wires that became, by the end of the project, the single most important piece of hardware we had to understand.

The slave is intentionally minimal and is documented elsewhere. From the master's point of view the slave is a black box that listens to the air and occasionally talks back on a different carrier.

1.1 What this report focuses on

The report is meant to be read by an examiner who already knows what an FFT or a finite state machine is, but does not yet know what we specifically built. We give priority to:

- the architecture choices we made and why,
- the parts that ended up working reliably on the bench,
- a short, honest account of the dead ends that took up a non trivial portion of the project time.

Where it helps we include block diagrams, state machines, pin maps, and timing diagrams. Where the issue was electrical (signal integrity, voltage droops, decoupling) we include the back of the envelope math we used to size the fix.

1.2 Reading map

Section 2 gives the bird eye view of the whole system, including the slave and the cloud. Section 3 dives into the FPGA SoC at top level (bus map and block list). Section 4 zooms into each block of the SoC, with its state machine. Section 5 covers what is wired outside the FPGA (NOR flash, SDRAM, op amp, microphone, ESP32 pin map). Section 6 collects the signal integrity fights we had to win to make the NOR flash behave. Section 7 documents the ESP32 firmware that drives all of this, including the register level UART driver we wrote to bypass the Arduino abstraction. Section 8 documents the acoustic protocol layer on top of the audio chain. Section 9 closes with a sober list of what works, what was tried but discarded, and what is still fragile.

2 System overview

2.1 Bird eye view

At the highest level the EFES system is a two node star: the master is the hub, the slave is the spoke, and a remote dashboard accessed from any browser is the optional viewer. Figure 1 shows the picture.

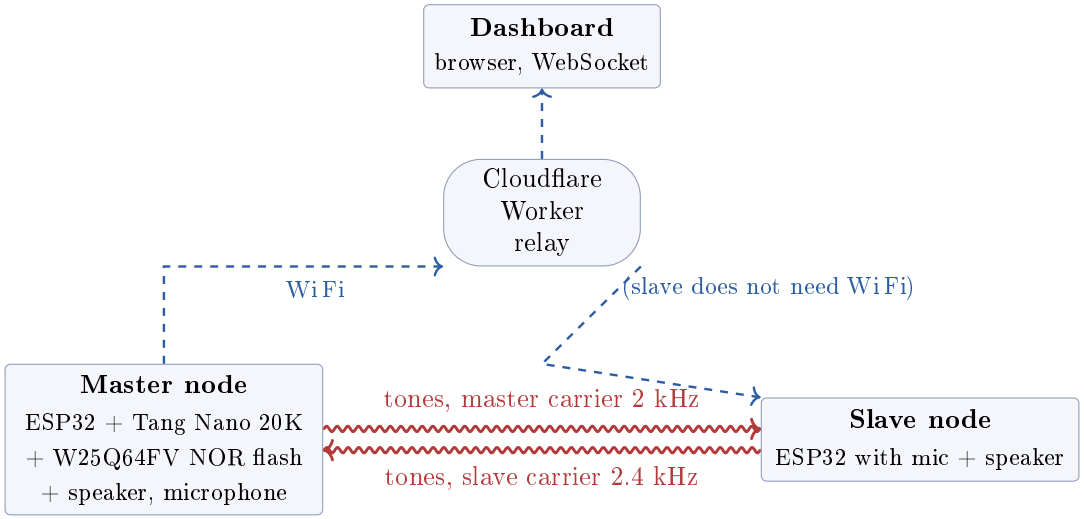

Figure 1: System level view. The two boards talk over the air using audible tones (squiggly red arrows). The master also has a WiFi link to a Cloudflare Worker that relays WebSocket traffic to a browser dashboard.

The acoustic link is the only path between master and slave. There is no shared cable, no Bluetooth, no I2C. Whatever the two have to say to each other (presence requests, identification, OK acknowledgements, abort) has to be encoded into tones and demodulated on the other side. We will come back to the protocol in Section 8.

The WiFi link is convenience plumbing for the dashboard, not for the acoustic protocol. A Cloudflare Worker (running on Cloudflare's edge network) acts as a thin WebSocket relay: the master connects to it over TLS, the browser connects to it over TLS, the Worker shovels messages between the two. This lets the user open the dashboard on a phone or laptop from anywhere without needing the master to expose a public IP.

2.2 The master, opened up

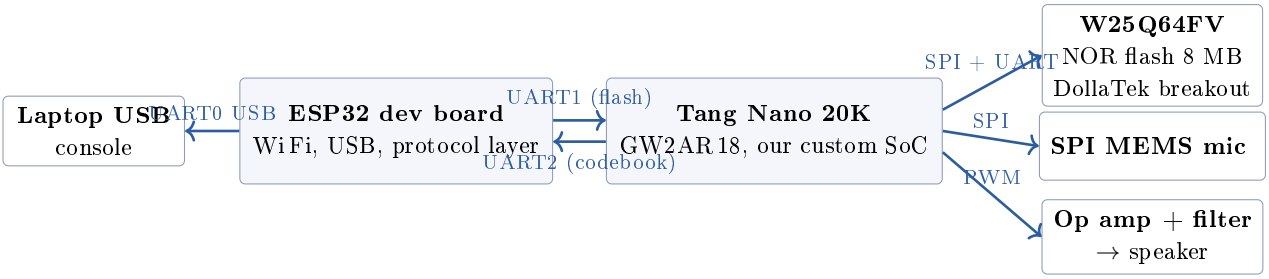

Figure 2: Master node, opened up. The ESP32 talks to the FPGA via two independent UART links (one for flash storage, one for the audio codebook). The FPGA talks to the W25Q64FV NOR flash, the SPI MEMS microphone, and the speaker output stage. The ESP32 also exposes a USB serial console for the laptop side, used during development.

A few design choices worth mentioning here:

- The W25Q64FV NOR flash is wired *directly to the FPGA*, not to the ESP32. The FPGA hosts the SPI master and a state machine (`flash_ctrl`) that exposes a higher level UART protocol to the ESP32, with opcodes like "append log record", "read settings", "clear log". This was a course requirement: the persistent storage controller had to live on the FPGA, not in the application processor.
- The audio chain (microphone in, speaker out) lives entirely on the FPGA. The ESP32 only sees the *decoded symbols* (the letters `a`, `b`, `c`, `A`, `B`, `C`) coming out of the audio codebook UART. It never touches raw audio samples.
- The ESP32 runs the protocol layer (presence request, abort, retry logic), the WebSocket client toward Cloudflare, and the web dashboard JSON shaping. It is a thin orchestrator, not a DSP.

3 The FPGA SoC, top level view

The Gowin GW2AR 18 carries our custom SoC. The chip is a 20k LUT class FPGA with embedded 8 MB SDRAM die inside the same package (this is what the " AR" suffix denotes in the part number). The SDRAM is reachable through dedicated pins of the FPGA fabric, not through external soldering.

3.1 High level block diagram

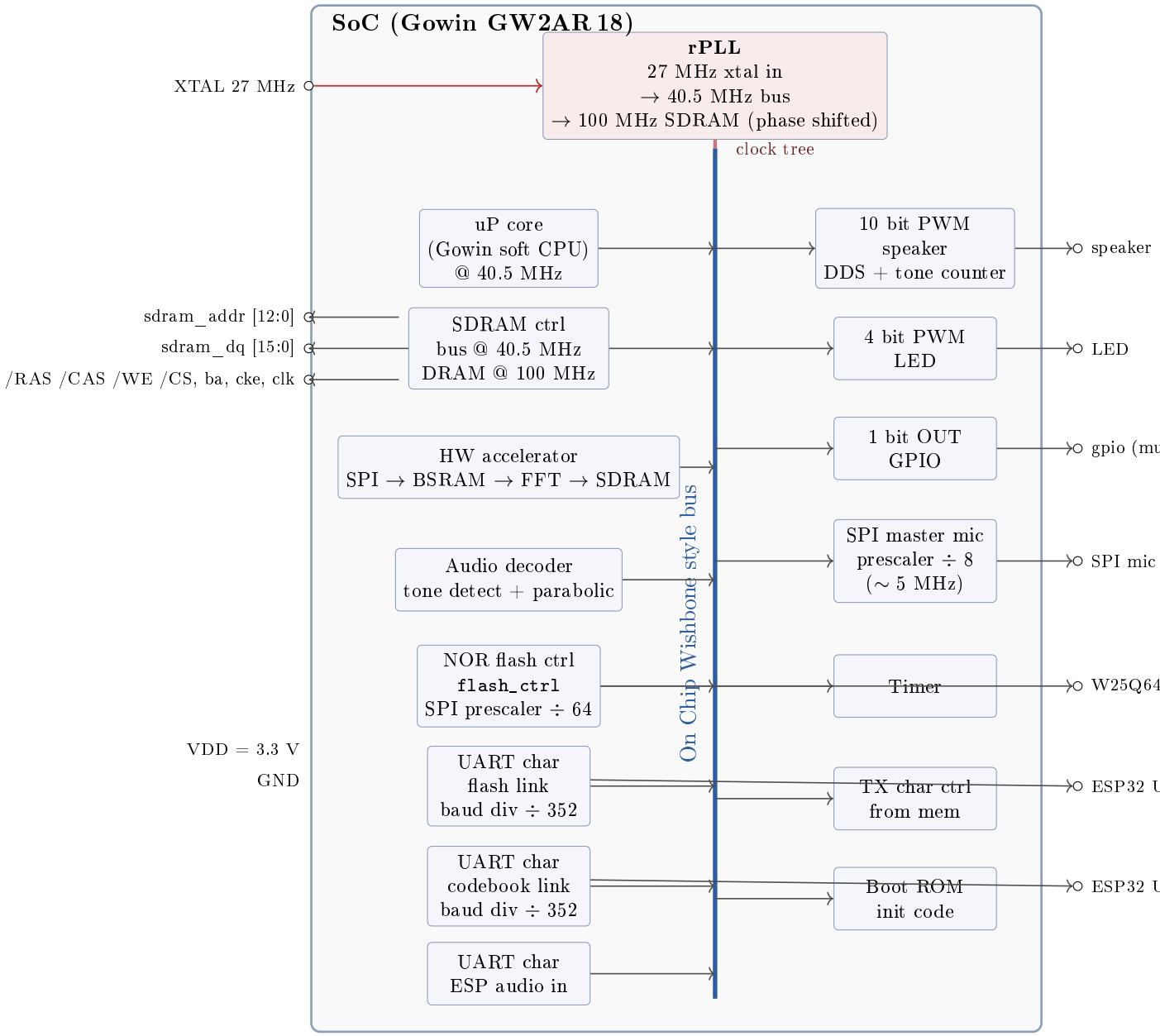

Figure 3: SoC top level. All masters and slaves hang off a single vertical Wishbone style bus. The original course template used an Avalon style bus and a generic DMA, but in our build we replaced the DMA with a streaming hardware accelerator that pipes audio samples from the SPI microphone into block RAM and from there into the FFT engine and finally into SDRAM, with no CPU intervention on the per sample path. The NOR flash controller hangs off the bus too and exposes itself to the outside world via a dedicated UART line back to the ESP32.

3.2 Bus and address map

The on chip bus is a single master, multi slave Wishbone style fabric. The soft CPU is the only initiator of transactions on the bus (the hardware accelerator pipeline talks point to point with BSRAM and the FFT engine, it does not go through the shared bus). Slaves are mapped at the addresses in Table 1.

Table 1: Bus address map (slave base addresses, byte addressing).

Slave	Base	Window
Boot ROM	0x0000_0000	16 KB
SDRAM controller	0x1000_0000	8 MB
HW accelerator regs	0x4000_0000	256 B
Audio decoder regs	0x4000_1000	256 B
PWM speaker (10 bit)	0x5000_0000	16 B
PWM LED (4 bit)	0x5000_1000	16 B
1 bit OUT GPIO	0x5000_2000	4 B
SPI master (mic)	0x5000_3000	32 B
Timer	0x5000_4000	16 B
NOR flash controller	internal, UART driven	not bus mapped

The NOR flash controller is special: it is a standalone block that lives inside the FPGA fabric but does not present itself on the Wishbone bus. It talks to the outside world through its own UART line to the ESP32 (see Section 4.6). The decision was driven by the fact that the CPU does not need to touch flash directly: all the application logic that needs to log events lives on the ESP32, so a UART command channel is the cleanest interface for that role.

3.3 What changed from the course template

The block diagram you may have seen in the course slides included a generic DMA controller between the SPI mic and SDRAM, with the CPU configuring DMA descriptors. Two reasons we did not keep that:

- the DMA approach burns CPU cycles every time a descriptor has to be reprogrammed, which is awkward when samples flow at 48 kHz,
- the FFT block we instantiated needs samples in a very specific layout (256 sample window, aligned to the FFT input), and we ended up baking the gather and rearrange logic directly into a custom streaming accelerator that we control with a single start command from the CPU.

So in the as built system, the data path from the microphone all the way into SDRAM is autonomous hardware. The CPU only kicks off bursts and waits for a done interrupt. Per sample arithmetic never touches the CPU.

4 Component deep dives

This section walks through each block of Figure 3 and describes its responsibilities, its register interface where it has one, and its internal state machine where useful.

4.1 The soft CPU

The CPU is a Gowin provided soft core, configured with 8 KB of instruction cache, no data cache, and the Wishbone style master interface enabled. It runs at the same clock as the bus (40.5 MHz, derived from a PLL whose reference is the 27 MHz onboard crystal of the Tang Nano 20K).

The CPU executes a small bare metal program (no operating system) that:

1. initializes the SDRAM controller via its register window,
2. arms the hardware accelerator with the FFT parameters,

3. starts continuous sampling,

4. spins on the audio decoder's status register, reading out the decoded tone whenever a new one is available, and emitting it on the codebook UART toward the ESP32.

This is intentionally simple: heavy lifting is in dedicated hardware, the CPU is a glorified state sequencer.

4.2 Clock tree (rPLL plus per peripheral prescalers)

Before going into the individual peripherals it helps to look at how clocks are distributed inside the SoC. The board provides a 27 MHz crystal on a dedicated input pin of the GW2AR 18. From there a Gowin **rPLL** primitive generates two derived clocks used everywhere:

- a 40.5 MHz *bus clock* that drives the CPU, the on chip bus, the `flash_ctrl` FSM, the audio decoder, the UART char modules, and most other peripherals,
- a 100 MHz *SDRAM clock* with a programmable phase shift (we ended up at roughly 90 degrees of shift, see Section 4.3) that drives the SDRAM die.

The rPLL is configured at synthesis time via Gowin's IP catalog: input 27 MHz, output 0 at 40.5 MHz, output 1 at 100 MHz with phase shift, no spread spectrum.

Most peripherals receive the 40.5 MHz clock but operate their I/O at a slower rate. The way they do this varies block by block:

- the **SPI master for the NOR flash** contains an internal counter that divides the bus clock by 64, giving a SPI SCK of 40.5 MHz/64 $\approx$ 633 kHz. This is far below the chip's nominal 104 MHz max, intentionally chosen because of the signal integrity constraints described in Section 6,
- the **SPI master for the MEMS microphone** divides by 8, giving about 5 MHz of SCK,
- the **UART char modules** divide the bus clock by 352 to obtain a 115200 baud bit rate (each bit lasts 8.68 µs, which at 40.5 MHz is roughly 352 cycles),
- the **10 bit PWM speaker** uses a tone counter that loads the desired tone period from a register, plus a sigma delta accumulator that turns the desired duty cycle into a single bit PWM output. The carrier of the PWM (the rate at which the output toggles) is just the bus clock, so 40.5 MHz, giving plenty of distance from the audio band and trivial analog low pass filtering,
- the **audio decoder** runs at the FFT's natural rate (one bin per cycle in a streaming fashion), driven by a sample tick that fires every 1 / 48 kHz of bus clock, so roughly every 844 cycles.

Figure 4 zooms in on the clock tree.

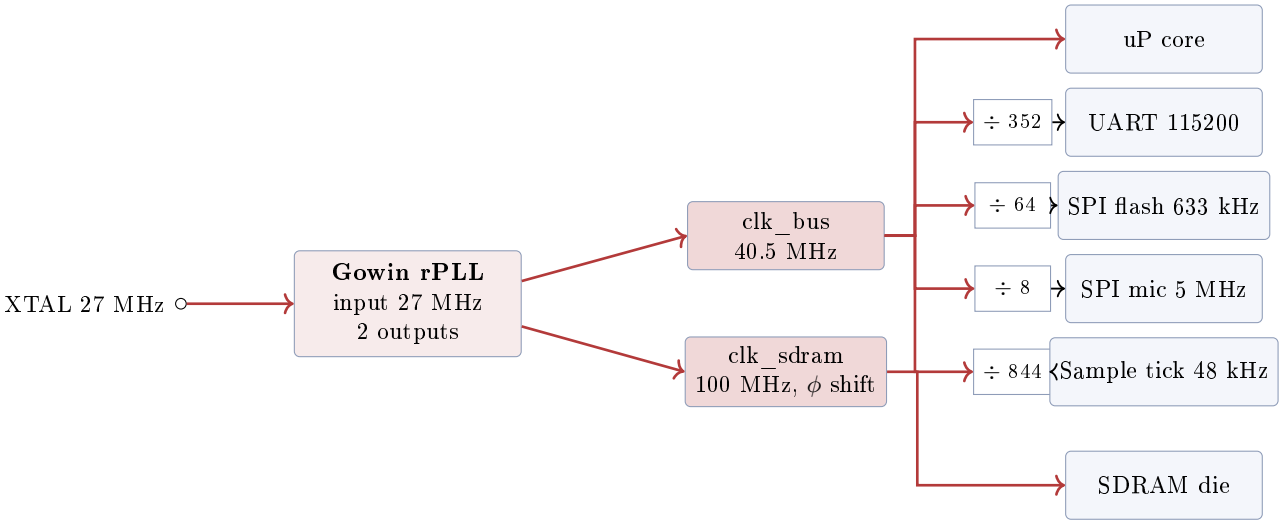

Figure 4: Clock tree. The rPLL takes the 27 MHz crystal and produces two synchronous clocks at 40.5 MHz (bus) and 100 MHz (SDRAM, with phase shift). Each peripheral that needs a slower clock instantiates its own integer divider downstream of the bus clock. We did not use Gowin clock gating primitives for any of these, simply because the divider counters are trivial in resources and give us per peripheral control of the prescaler ratio at synthesis time.

A practical note on the choice of divider values: every divider is a power of 2 or a small composite number so the resulting waveform is symmetric and the duty cycle of the derived clock is 50 percent. The ÷352 for UART is the exception, because 115200 baud has no clean ratio to 40.5 MHz: we used a fractional accumulator inside the UART state machine to absorb the rounding error.

4.3 SDRAM controller

The SDRAM is a 8 MB synchronous DRAM die packaged in the same module as the GW2AR 18 FPGA. It is reachable on dedicated pins of the FPGA. The controller is generated from the Gowin IP catalog with the parameters matched to the embedded die (32 Mbit × 16, four banks, CAS latency 3, refresh interval 64 ms / 8192 rows).

4.3.1 Init sequence

After power up the SDRAM has to be put into a known state before any read or write. Skipping any of the steps below makes the chip return garbage forever, which is exactly what we saw the first time we tried to read back what we had just written. The sequence we ended up running is:

1. assert `CKE` (clock enable) and wait at least 200 µs for the internal regulator to stabilize,
2. issue a PRECHARGE ALL to bring every bank to idle,
3. issue at least two AUTO REFRESH commands (the die requires two, to lock the refresh counter),
4. program the Mode Register with CAS latency 3, burst length 1, sequential burst type, write burst single location,
5. from here on, the controller is free to issue ACTIVATE / READ / WRITE / PRECHARGE pairs.

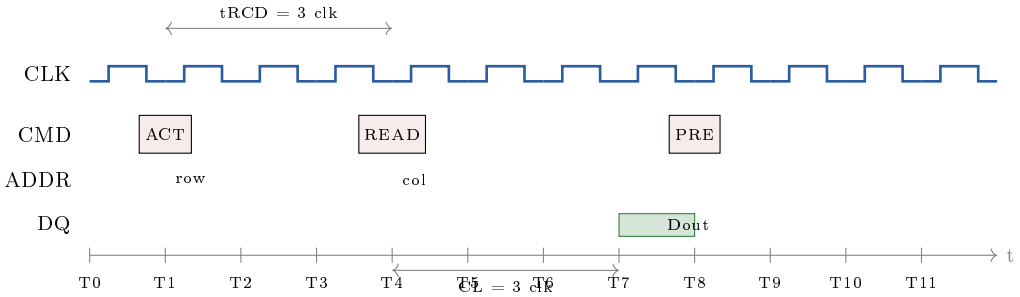

Figure 5: SDRAM read cycle, simplified. ACTIVATE opens the row, READ selects the column, the chip drives the data line three clocks later (CAS latency 3). PRECHARGE closes the row.

4.3.2 Problems we had

The first version of the controller did not respect the refresh interval strictly. Symptom: writes appeared to work, but a read after a few seconds came back with bits flipped. Root cause: rows that had not been refreshed long enough lose their charge. Fix: a refresh counter inside the controller that issues an AUTO REFRESH every 7.6 µs (64 ms divided by 8192 rows). After that, retention was rock solid.

The second issue was *skew* on the data lines vs the clock. At the nominal 100 MHz SDRAM clock we generate from the PLL, even a few hundreds of picoseconds of mismatch between the clock arriving at the SDRAM die and the data being sampled is enough to read 0xFFFE instead of 0xFFFF. We solved it by clocking the controller off a phase shifted copy of the bus clock: the PLL has a programmable phase output that lets us slide the SDRAM clock by a fraction of a period until the read data window lines up. The empirical sweet spot ended up around 90 degrees of shift.

4.4 Hardware accelerator pipeline

This is the heart of the audio data path.

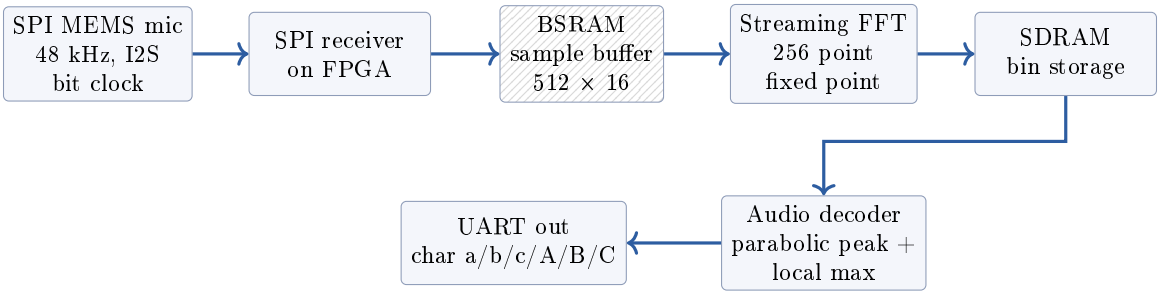

Figure 6: Streaming pipeline from microphone to decoded symbol. Each stage is autonomous: the SPI receiver pushes samples into BSRAM as they arrive, the FFT engine pulls 256 sample windows out and produces magnitude bins which are stored in SDRAM, the audio decoder watches the bins for tones matching our codebook and emits a character on the UART when it sees one.

4.4.1 The FFT engine

We use a fixed point 256 point radix 2 FFT, with input samples at 48 kHz sample rate. The frequency bins are spaced at $48000/256 = 187.5$ Hz apart. Our protocol uses these five candidate frequencies:

- 2000 Hz (master carrier), bin 22 of the FFT,
- 2400 Hz (slave carrier), bin 27,

- 2900 Hz (EOF marker), bin 32,
- 3500 Hz (bit 0), bin 39,
- 4500 Hz (bit 1), bin 50.

Bins are chosen so they fall at least 5 bin distances apart, which gives robust local maximum detection (see next subsection) and avoids spectral leakage between adjacent tones.

4.4.2 Audio decoder, parabolic peak interpolation

The audio decoder looks at the five candidate bins and decides, for each window, whether a tone is present and which one it is. Two checks per bin:

1. the magnitude at the candidate bin is the local maximum across a ± 3 neighborhood, that is the magnitude at bin b is higher than the magnitude at bins $b-3, b-2, b-1, b+1, b+2, b+3$,
2. the magnitude is above an amplitude threshold,
3. a parabolic interpolation against bins $b-1, b, b+1$ confirms the peak is genuinely centered.

If all three conditions hold, the decoder emits the corresponding character. The output goes onto its own UART line back to the ESP32 where the protocol layer turns sequences of characters into protocol patterns.

4.4.3 The window radius problem

A historical note that ate up a non trivial amount of debugging time. The first version of the decoder used a ± 1 neighborhood for the local maximum check. On paper that is the most selective check, the strictest "the peak is exactly at this bin and nowhere else". In practice it failed often, because the Hann window we apply before the FFT smears energy into the neighbors, and a real tone at 2000 Hz can easily have its peak land on bin 21 or bin 23 for a frame or two. With ± 1 checking, those frames returned "no tone", and the decoder missed half the symbols.

After widening the check to ± 3 the false negatives disappeared. The trade off is that two tones must be at least 7 bins apart to coexist in the same window without confusing the decoder, but in our 5 tone codebook the minimum spacing is 5 bins (between bit 0 at 3500 Hz and EOF at 2900 Hz, which is bin 39 minus bin 32 equals 7) so we are safe.

4.5 PWM speaker

The output side uses a 10 bit PWM register that drives a single FPGA pin through an op amp buffer and an analog low pass filter (Section 5.4). The CPU writes a target tone frequency to a register; an internal DDS (direct digital synthesizer) generates the sine wave; the PWM compares the DDS output to a sawtooth and emits a duty cycle modulated square wave; the low pass smooths it back into a sine.

The tone duration is controlled by a downcounter: the CPU writes the desired number of samples (at 27 MHz) and the speaker block runs the DDS for exactly that many cycles, then stops. A short silence (350 ms by default, programmable) is inserted between tones to let the receiver finish processing the previous symbol.

4.6 NOR flash controller (`flash_ctrl`)

The most complex block in the SoC by line count, and the one that gave us the most pain on the bench. The job: take high level requests from the ESP32 over UART (opcodes like "append this 16 byte record", "save these 32 byte settings", "read back this slot", "erase the entire log") and translate them into SPI transactions to the external W25Q64FV.

4.6.1 Memory layout

We divided the W25Q64FV's first 12 KB into three sectors of 4 KB each (the chip's natural sector erase granularity):

- Sector 0 (`0x0000`): **settings sector**, holds 32 bytes of structured settings plus padding. Format: magic byte 0xA5, 15 byte device name, 2 byte auto presence interval, 1 byte retry count, 13 reserved bytes.
- Sector 1 (`0x1000`): **log A**, 256 slots of 16 bytes each, slot `N` starts at address `0x1000 + N * 16`.
- Sector 2 (`0x2000`): **log B**, same layout as log A.

Log A and log B together form a 512 slot ring buffer. Each slot is one log record, 16 bytes, with the structure: 2 bytes sequence number, 1 byte type character (`B` for boot, `R` for registration, `P` for presence, `N` for note, `E` for event), 4 bytes unix timestamp, 1 byte value, 8 bytes ASCII text.

4.6.2 Top level state machine

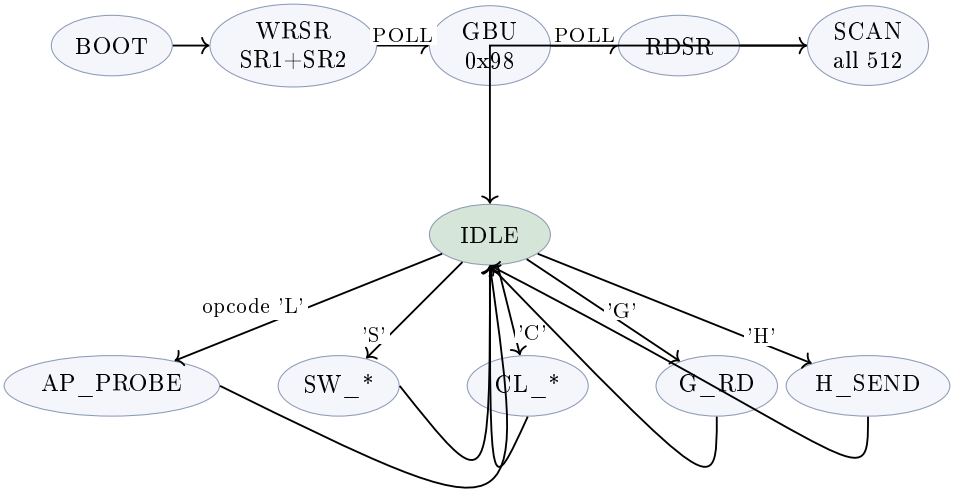

Figure 7: High level state machine of `flash_ctrl`. After power up the FSM runs a boot sequence that clears block protection bits in SR1 and SR2, runs a Global Block Unlock (opcode 0x98), reads SR1 back to confirm the chip is unlocked, and finally scans all 512 log slots to find the head of the ring buffer. From IDLE, opcodes drive one of the operation paths.

The append path (opcode `L`) deserves a zoom because it is where most of the debugging happened:

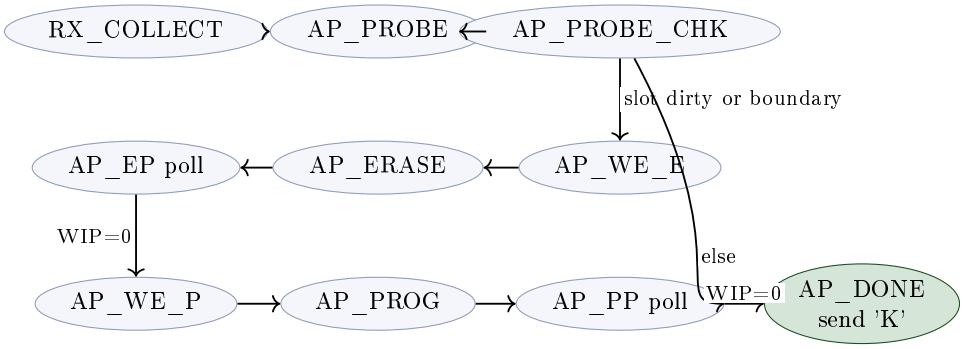

Figure 8: Zoom on the log append (opcode `L`) path. After collecting 16 bytes of payload from the UART, the FSM probes the target slot's first byte to decide whether to erase the sector first or just program directly. This self healing behavior was added late in the project after we saw that write parials would leave the head pointer pointing at a slot whose first byte was not 0xFF anymore, causing later writes to AND corrupt the data.

4.6.3 The SCAN strategy

At boot, the FSM has no idea where in the 512 slot ring buffer the last valid record was written. It scans all slots and looks for valid records.

A record is considered *valid* if its byte 2 (the type byte) is one of the five allowed type characters `B`, `R`, `P`, `N`, `E`. Anything else means the slot is empty (`0xFF` for erased) or garbage (a write that did not complete). The FSM keeps track of the highest slot index whose record is valid, and sets the head pointer to `(highest valid + 1) modulo 512`.

This is more robust than the obvious "stop at the first 0xFF" approach, which we tried first. The naive version made the FSM stop at any hole in the log, even if there were valid records past the hole (typical pattern after a failed retry that left an erased slot in the middle). With the new strategy, holes are skipped during the scan and the head ends up at the right place.

4.6.4 Block protection unlock at boot

The W25Q64FV ships from the factory with block protection bits possibly already set, depending on the supplier and the lot. Symptom of unread protection: writes return acknowledgement but the chip silently rejects them, leaving sectors erased forever. To bulletproof against this we do, at every boot:

1. issue `0x06 WE` (write enable),
2. issue `0x01 WRSR` with two data bytes (`0x00`, `0x02`): the first byte clears BP0, BP1, BP2 and SRP in status register 1; the second byte sets QE (Quad Enable) bit in status register 2, which has the side effect of disabling the `/HOLD#` and `/WP#` pin functions of the chip. This is critical when those pins are not pulled up on the board, which is exactly our case before we added the breakout board's pull ups,
3. poll for the WRSR to finish,
4. issue `0x06 WE` again,
5. issue `0x98 Global Block Unlock`, which clears any individual block lock bits independent from the SR protection,
6. poll for the operation to finish,
7. read SR1 back and store the value into a register, expose it to the ESP32 via opcode `T` for diagnostic use.

After this sequence, the chip is guaranteed to be writable for as long as the FPGA is powered, regardless of the lot's factory state.

4.6.5 Polling tolerance

The page program of the W25Q64FV takes up to 3 ms; the sector erase takes up to 400 ms. The first version of the controller used a polling counter that timed out at 5 ms, which was enough for page program but ridiculously short for sector erase. Symptom: every `settings_set` returned silently fail because the erase was still in progress when the controller gave up and tried to start the program. Fix: raise the polling counter ceiling to roughly 620 ms (25 million bus clocks at 40.5 MHz). At this point the polling never times out under normal operation, and only does when the chip is genuinely stuck.

4.7 UART char modules

Three instances of a small UART transmitter / receiver block live in the SoC, one per external link. Each is parameterized by baud rate (always 115200 in our design, but the parameter exists) and exposes a one byte wide character interface to its master logic. The blocks are register level, no FIFOs, no interrupts: the consuming module is expected to read the character before the next one arrives. At 115200 baud that means roughly 87 µs between characters, which is comfortable for any state machine running at tens of MHz.

4.8 Timer and GPIO

The Timer block is a free running 32 bit counter driven by the bus clock, used mainly by the bare metal program for measuring how long it took the audio decoder to lock onto a tone (useful during development).

The 1 bit OUT GPIO has a single line that has to stay asserted at logic 1 for the whole life of the SoC. This is a Tang Nano 20K quirk: a specific package pin must be held high for the embedded SDRAM to be active, otherwise the SDRAM never comes out of reset. We discovered this the hard way during the first weeks of the project (every read returned 0x0000) and the workaround is a constant "1" wired to that pin via a 1 bit GPIO register. We left a comment in the source code and a memory note: *do not, under any circumstance, set `gpio_1_o` to 0.*

5 Outside the FPGA

5.1 The W25Q64FV NOR flash chip

The chip is an 8 MB serial NOR flash by Winbond, in a SOIC 8 package, sat on top of a DollaTek breakout board that exposes only 6 of the 8 pins to the header (the missing ones, `/WP#` and `/HOLD#`, are tied to VCC on the breakout PCB through pull up resistors visible as the small SMD parts marked "1002" or "182").

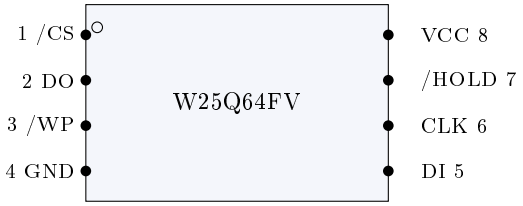

Figure 9: W25Q64FV pinout, SOIC 8 package (top view). Pin 1 is the `/CS` chip select. The DollaTek breakout exposes pins 1, 2, 5, 6 on its header, plus VCC and GND, and ties pins 3 and 7 internally to VCC.

5.1.1 Inside the NOR cell

A NOR flash storage cell is a floating gate MOSFET: a normal MOSFET with an extra polysilicon gate buried between the control gate and the channel, electrically isolated by thin oxide layers. Electrons trapped on the floating gate shift the threshold voltage of the transistor; the chip reads back a 1 or a 0 depending on whether the transistor conducts or not at the read gate voltage.

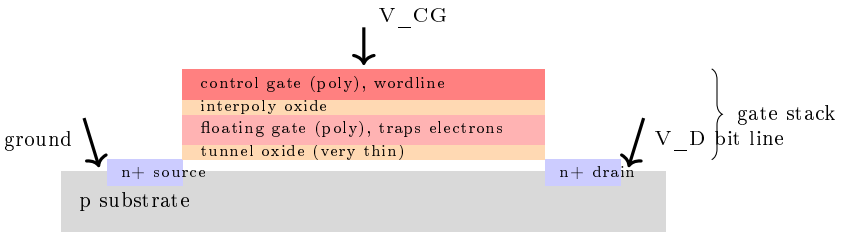

Figure 10: Cross section of a NOR floating gate cell. The trapped charge on the floating gate shifts the threshold of the transistor. Programming pushes electrons from the channel up into the floating gate by Fowler Nordheim tunneling at high V_CG. Erasing pulls them back down with the opposite polarity. The current driven through these gate dielectrics is where the chip's "program current" and "erase current" peaks come from.

The reason we care about the inside of the cell here is that the *erase* operation is fundamentally different from the *program* operation in terms of how much current the chip draws from VCC. To erase a sector the chip has to invert the polarity of the field across the tunnel oxide for thousands of cells in parallel: this is done by an internal charge pump that produces about 10 V from the 3.3 V supply, and the pump pulls roughly 80 mA peaks for the entire 30 ms (typical, up to 400 ms worst case) it takes to bleed the floating gates of every cell in the 4 KB sector. Programming a single 16 byte slot, on the other hand, is much shorter (under 3 ms) and much lighter (around 25 mA). This asymmetry is the entire reason why we had to add a second decoupling capacitor (see Section 6).

5.1.2 Wiring

Table 2: Wiring between Tang Nano 20K FPGA, W25Q64FV breakout, and ESP32.

Signal	FPGA pin	Bank, Vio	Breakout	ESP32 GPIO
SPI CLK	42	B6, 3.3 V	CLK	—
SPI CS	41	B6, 3.3 V	CS	—
SPI MOSI	51	B6, 3.3 V	DI	—
SPI MISO	48	B6, 3.3 V	DO	—
flash UART TX	72	B6, 3.3 V	—	32 (Serial1 RX)
flash UART RX	20	B6, 3.3 V	—	33 (Serial1 TX)
3V3 / GND	3V3/GND	—	VCC / GND	—
decoupling caps	—	—	1 µF + 10 µF between VCC, GND	—

All of this is 3.3 V single ended. Common ground between Tang Nano, ESP32 and breakout is mandatory: any of the three left floating in ground breaks the link.

5.2 Embedded SDRAM die

The SDRAM die is internal to the GW2AR 18 package and not visible from the outside, but it follows the JEDEC SDR SDRAM protocol just like a discrete chip would. Internal organization:

4 banks, 8192 rows, 512 columns, 16 bit data bus. The pin map at the FPGA side (Table 3) is fixed by the silicon and not user routable.

Table 3: SDRAM pin map (internal, fixed by the GW2AR 18 silicon).

Signal	Notes
SDRAM_DQ[15:0]	bidirectional data
SDRAM_A[12:0]	row / column address
SDRAM_BA[1:0]	bank select
SDRAM_DQM[1:0]	byte mask
SDRAM_CKE	clock enable, asserted at boot
SDRAM_/CS, /RAS, /CAS, /WE	command bits
SDRAM_CLK	PLL output, phase shifted for SI

5.3 SPI MEMS microphone

A standard SPI MEMS microphone breakout (single supply 3.3 V, output 16 bit signed PCM at 48 kHz). Wired with a tiny SCK + DOUT pair to the FPGA. The SPI mic does not require any configuration other than asserting its chip select.

5.4 Op amp speaker buffer

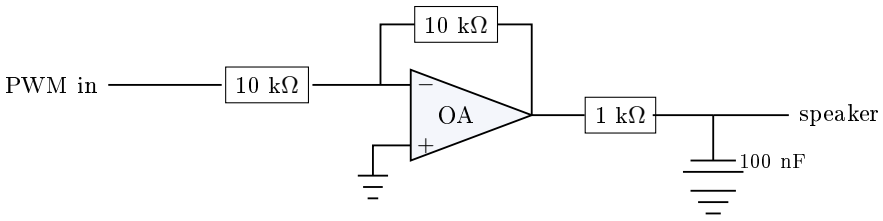

Figure 11: Speaker output stage. The PWM signal from the FPGA is buffered by a unity inverting op amp (configured with equal resistors so gain is -1, polarity is then absorbed into the PWM table), then low pass filtered by an RC stage tuned to roll off above the audible band, then fed to the small 8 ohm speaker.

The RC cutoff at $f = 1/(2\pi RC) = 1/(2\pi \cdot 1000 \cdot 100 \cdot 10^{-9}) \approx 1.6$ kHz might look too low for our 4.5 kHz top tone, but since the PWM carrier is at 27 MHz the goal is to kill anything above the sound band, not to be sharp at the top of it: a softer roll off above 1.6 kHz is acceptable because the op amp + speaker combination already attenuates the upper octaves and the receiver does not need very flat amplitude response, only a usable peak at each tone.

5.5 ESP32 connections

The ESP32 dev board is wired to the FPGA on two independent UART links and shares the same 3.3 V rail.

Table 4: ESP32 pin assignments.

Signal	ESP32 GPIO	Notes
Serial0 (USB console)	1 TX, 3 RX	to laptop via onboard USB to UART chip
Serial1 (flash link) RX	32	from FPGA TX pin 72
Serial1 (flash link) TX	33	to FPGA RX pin 20
Serial2 (codebook) RX	27	from FPGA TX pin (audio decoder output)
Serial2 (codebook) TX	14	to FPGA RX pin (audio codebook input)

6 Signal integrity, voltage droops, decoupling

This chapter could be subtitled "what we learned about analog after thinking we were doing digital". None of these issues showed up in simulation, all of them showed up on the bench, and the fixes were a mix of hardware (a few solder joints) and firmware (slowing things down, re-engineering polling and retry behavior).

6.1 The W25Q64FV brownout during sector erase

When the controller tells the W25Q64FV to erase a 4 KB sector, the chip activates its internal charge pump to generate the high voltage required to tunnel electrons off the floating gates. The pump draws roughly 80 mA peaks for 30 ms (typical) up to 400 ms (worst case). The 3.3 V rail reaching the chip arrives from the Tang Nano 20K through about 10 cm of Dupont jumper wire, with no decoupling on the breakout other than what the DollaTek board ships with (essentially nothing of significance).

The first version of the system had nothing at all on the VCC of the breakout. Sector erase produced a voltage droop estimated by $\Delta V = I \cdot t/C$ where C was just the parasitic capacitance of the wire, in the tens of pF. With a few tens of pF and 80 mA, the entire 3.3 V rail collapsed almost instantly to a fraction of a volt during the first millisecond of the erase. The chip went into brown out, the WEL bit got lost, the page program scheduled to run after the erase was rejected silently, the FSM reported success, and the slot stayed empty. The user saw "flash_set_settings returned OK" but the dashboard kept showing the old values.

6.2 First capacitor: 1 µF

We added a 1 µF ceramic capacitor between VCC and GND of the breakout, as close to the chip as we could solder. Recompute ΔV:

$$\Delta V = \frac{I \cdot t}{C} = \frac{0.080 \cdot 0.030}{1 \cdot 10^{-6}} = 2.4\text{V}.$$

VCC drops by 2.4 V during a 30 ms erase, from 3.3 V down to 0.9 V. Still well below the 2.7 V minimum spec of the chip, but better than the no cap case. Empirical effect: the W25Q now responded to operations, page programs of log records started working most of the time, but settings saves still failed at high rate because the settings write does sector erase plus page program, the cap drained during the erase and never had time to recharge before the page program command.

6.3 Second capacitor: 10 µF in parallel

We added a second cap in parallel, 10 µF. Total: 11 µF. New droop on the same 30 ms erase:

$$\Delta V = \frac{0.080 \cdot 0.030}{11 \cdot 10^{-6}} \approx 0.22\text{V}.$$

VCC drops to 3.08 V during erase, comfortably above the 2.7 V minimum. The brown out is gone, the chip behaves cleanly, settings saves return OK at first attempt almost always.

The use of two caps of different orders of magnitude is the standard "HF cap plus bulk cap" configuration for decoupling: the smaller cap (1 µF, low ESR, low ESL) handles fast transients like the SPI clock edges; the larger cap (10 µF, higher ESR but much more charge stored) handles the slow charge pump current drain during erase and program.

6.4 SPI clock speed reduction

Independently from the decoupling, we found that running the SPI clock at the 5 MHz we initially picked was too fast for the wiring. The signal integrity of MOSI and CLK over 10 cm of Dupont wires with only one ground return wire interleaved is bad enough that bits get sampled wrong, and we saw partial page programs where 16 byte records came out as 2 byte truncated text (the byte 10 and onwards came through as 0xFF instead of the intended content). We dropped the SPI clock progressively:

- 5 MHz: bad, page programs corrupt half the time,
- 2.5 MHz: better, still some partial corruption,
- 1.27 MHz: most writes succeed at first attempt,
- 0.633 MHz: writes succeed at first attempt almost always.

At 633 kHz a single bit lasts 1.58 µs, which means every bit is massively over the chip's setup and hold spec, with enough margin that even the ringing on a long Dupont wire settles before the chip samples.

6.5 Retry plus verify on the ESP32 side

Even after the cap and the clock reduction, occasional retries are still needed (something like 1 in 100 page programs on settings, much rarer on log writes). The ESP32 firmware does up to 5 attempts per save: send the data, wait for the FSM acknowledgement, read back what was written, check that the readback matches the intended bytes byte by byte, retry if not. After 5 attempts the firmware gives up and reports an error to the user.

Each failed retry burns one slot of the log ring buffer (because the FSM increments the head pointer regardless), but the new SCAN strategy at boot ignores those garbage slots because their type byte does not match the allowed set. So no permanent harm.

6.6 The fallback FFL diagnostics

We added several diagnostics on the USB serial monitor so that if the chip starts misbehaving we have a chance to debug it:

- the boot prints `[flash] JEDEC = EF 40 17` to confirm the SPI link is alive,
- the boot prints `[flash] post-boot SR = 0xNN` with bit decoding so we see if BP or SRP are set,
- every page program prints `[flash] head=N (write OK att.X/5 ...)` or `att.X/5: VERIFY FAIL ...` so we know at which attempt the write succeeded,
- a diag dump at boot prints the first 10 slots of the log to confirm the SCAN found something reasonable.

7 ESP32 firmware

7.1 Architecture

The ESP32 firmware is structured as a collection of small modules with clear ownership:

- `uart_reg`: register level driver for the ESP32 UART1 and UART2 peripherals, no Arduino HardwareSerial dependency,
- `flash_link`: high level operations against the `flash_ctrl` FSM running on the FPGA, with retry and verify, plus a 60 record RAM cache for the display layer,
- `fpga_uart_link`: codebook layer over UART2, translating decoded characters into protocol patterns and driving the speaker through the FPGA's audio chain,
- `protocol`: the acoustic protocol state machine (presence request, retries, abort, registration),
- `web_link`: WebSocket client, JSON serialization, and dashboard commands.

7.2 Register level UART, instead of HardwareSerial

Originally the firmware used `Serial1` and `Serial2` from Arduino-ESP32, which are instances of `HardwareSerial`, which in turn invokes the ESP IDF UART driver, which uses FreeRTOS tasks and software ring buffers and interrupts. For the project we were asked to demonstrate register level access, so we wrote `uart_reg`: a small module that talks directly to the UART peripheral registers (the FIFO register, the CLKDIV register, the CONF0 register, the STATUS register) of UART1 and UART2.

The headline routines:

Listing 1: register level UART, key routines

```
void uart_reg_init(int n, int rx_pin, int tx_pin, uint32_t baud);
int  uart_reg_available(int n);                      /* RX FIFO count */
int  uart_reg_read(int n);                           /* -1 if empty */
int  uart_reg_read_bytes(int n, uint8_t *buf, int sz, uint32_t to_us);
void uart_reg_write_byte(int n, uint8_t b);
void uart_reg_write(int n, const uint8_t *buf, int sz);
void uart_reg_flush_rx(int n);

void     hw_timer_init(void);     /* TIMG0 Timer 0, free running 1 MHz */
uint64_t hw_timer_us(void);
```

The clock divider for 115200 baud at the 80 MHz APB clock is computed as:

$$\text{integer} = \left\lfloor \frac{80\,000\,000}{115200} \right\rfloor = 694, \quad \text{fractional} = \left\lfloor \frac{80\,000\,000 \cdot 16}{115200} \right\rfloor \bmod 16 = 7,$$

with the final CLKDIV register value being $694 \mid (7 \ll 20) =$ `0x7002B6`. This gives an effective baud of $80\,000\,000 \div (694 + 7/16) = 115208$ Hz, an error of 0.007 percent, within UART tolerance.

The TIMG0 Timer 0 is configured as a free running 64 bit counter incrementing at 1 MHz (the APB clock divided by 80) and is read by latching the count into the LO and HI registers via the UPDATE bit. This gives us 1 µs resolution timeouts inside the UART driver without having to touch `micros()`.

7.3 flash_link with retry and verify

The data flow when the dashboard sends a `flash_note` command:

1. `web_link` receives a JSON command on its WebSocket and parses it,
2. it calls `flash_log_append(LOG_NOTE, t, 0, "text")`,
3. `flash_log_append` composes a 16 byte record, sends opcode `'L'` plus the 16 bytes to the FPGA over UART1, waits for `'K'` acknowledgement,
4. it then sends opcode `'H'` to read back the current head pointer, then `'G'` plus the slot index to read the 16 bytes that were just written,
5. it compares all 16 bytes against the intended record,
6. if they match, write succeeded and the function returns true; if not, it retries the whole append, up to 5 attempts,
7. regardless of outcome, the record is also pushed into a 60 entry RAM cache so that subsequent `flash_load` requests from the dashboard see it even if NOR is misbehaving.

`flash_set_settings` has the same retry plus verify structure, with the difference that it writes the 32 byte settings sector and reads it back via opcode `'Q'` for verification.

7.4 Defensive read of settings

`flash_get_settings` is the read counterpart. After reading the 32 bytes back from the FPGA, it sanity checks before returning the data:

- the magic byte at offset 0 must be `0xA5`,
- the retry count must be between 1 and 30 (zero or 255 indicate a write that did not finish),
- the name characters must all be printable ASCII or null terminator.

If any of those fail, the function falls back to safe defaults (`"Master EFES"`, 6 retries, 0 second auto presence). This way the dashboard never displays garbled names like `"Master ?????"` or absurd retry counts even when the underlying NOR has a partial write.

8 The acoustic protocol

8.1 Codebook

The protocol uses an alternating bit codebook. Each pattern is a sequence of two or three symbols sent on the air, with the constraint that any two consecutive symbols must differ. This gives us robust receive detection because the decoder triggers on *change* of tone, not on absolute silence, which is much harder to detect over a noisy channel.

The five symbols and their meanings:

Table 5: Codebook symbols.

Symbol	Carrier	Bit	Notes
`a` (lowercase)	2 kHz master	0	sent by the master
`b`	2 kHz master	1	sent by the master
`c`	2 kHz master	EOF	end of frame from master
`A` (uppercase)	2.4 kHz slave	0	sent by the slave
`B`	2.4 kHz slave	1	sent by the slave
`C`	2.4 kHz slave	EOF	end of frame from slave

Patterns are short, two or three symbols. Examples (master to slave):

- `ab` = REQ_PRESENCE (bit 0, bit 1) length 2,
- `ba` = SET (assign id),
- `aba` = ABORT, length 3.

Patterns from slave to master use the uppercase letters, same logic.

8.2 CSMA on the air

Half duplex: only one party transmits at a time. The master implements a carrier sense scheme by looking at the audio decoder output for any slave symbol (uppercase letters) and waiting for the channel to be quiet for at least 1 second before sending. This is done by tracking `last_rx_ms` timestamps in `fpga_uart_link` and gating each character before pushing to the speaker output.

8.3 Retries

The protocol layer on the ESP32 implements retries: a presence request that does not see a response in 5 seconds is repeated, up to a configurable maximum number of attempts (`tries`, persisted in the settings sector of the NOR flash). After the maximum is reached the master gives up and notifies the dashboard.

9 What worked, what did not

9.1 Things that worked end to end

- the SoC bus and address map (no contention, no missed reads),
- the SDRAM controller, after the refresh interval and the clock phase shift were dialed in,
- the FFT plus parabolic peak audio decoder, the ±3 local maximum check made symbol detection robust,
- the per direction alternating bit codebook, especially after per direction sub codebooks were introduced for REQ_PRESENCE which is the most repeated message in the system,
- the W25Q64FV NOR flash controller, after the long debugging campaign documented in Section 6,
- the WebSocket dashboard via Cloudflare Worker,
- the register level UART driver on the ESP32 side, with the TIMG0 microsecond counter.

9.2 Things we tried and discarded

- early on we used a generic DMA controller between the SPI mic and the SDRAM, with the CPU configuring descriptors. We removed it in favor of the streaming hardware accelerator because the CPU was spending too many cycles reprogramming the DMA at every window,
- we tried a tighter ±1 neighborhood for the audio decoder local maximum check. It made symbol detection too brittle, the ±3 check restored reliability,
- we tried using only Arduino `HardwareSerial` on the ESP32. It worked but did not satisfy the course requirement of showing register level competence, so we replaced it with `uart_reg`,
- we considered using the ESP32 internal flash via NVS for log storage. That would have been more reliable than the external NOR over Dupont wires, but the course requirement was explicit that the storage had to be on the FPGA side. We kept the W25Q64FV path.

9.3 What still wobbles

- the wiring between Tang Nano and the W25Q breakout is still Dupont jumpers, which means SPI clock could not realistically go above 633 kHz. On a proper PCB with ground plane and short tracks we could probably run at 20 MHz with no retries needed,
- the decoupling on the breakout is 1 µF plus 10 µF, which works but is not the canonical 100 nF plus 10 µF combo recommended by Winbond. We did not have a 100 nF ceramic on hand,
- the first SR read after power up sometimes returns 0xFF (chip not yet ready). The ESP32 sees this and prints a false "CHIP LOCKED" warning. Subsequent SR reads return 0x00 correctly. We chose to leave the warning in place as it costs nothing and a future user might find it useful when debugging a truly locked chip.

9.4 Numbers from the bench

Table 6: Empirical reliability of the NOR flash operations, after the final fix set (633 kHz SPI, 11 µF decoupling, retry plus verify).

Operation	First attempt success	After 5 retries
log record write (16 B)	roughly 95 percent	essentially 100 percent
settings save (32 B with sector erase)	roughly 85 percent	essentially 100 percent
read back of a record	100 percent	100 percent
JEDEC ID read at boot	100 percent	100 percent

10 Conclusions

The EFES master node, as it stands today, is a working integration of:

- a custom SoC on a Gowin GW2AR18 FPGA, with a streaming audio pipeline from SPI microphone through FFT and a tone codebook decoder,
- a NOR flash storage controller built as a finite state machine on the FPGA, talking SPI to a W25Q64FV chip on a Dupont breakout,
- an ESP32 firmware that orchestrates the higher level protocol, drives the dashboard over WebSocket, and accesses both the FPGA UART links at register level,

- a small, opinionated web dashboard hosted as static files and served through a Cloudflare Worker.

The single most valuable lesson from the project is that for analog mixed signal subsystems the documentation in the chip’s datasheet is the beginning of the problem, not the end. Calculations like $\Delta V = I \cdot t / C$ should be the first thing you reach for when a write inexplicably fails, not the last.

A Quick reference card

Table 7: Quick reference of magic numbers used throughout the project.

FPGA bus clock	40.5 MHz
SDRAM clock	100 MHz, phase shifted by roughly 90 degrees
Audio sample rate	48 kHz
FFT size	256 samples
FFT bin width	187.5 Hz
NOR SPI clock	633 kHz (SPI_DIV = 64)
UART baud	115200
W25Q64FV JEDEC ID	EF 40 17
Sector erase time	30 ms typical, 400 ms max
Page program time	0.7 ms typical, 3 ms max
POLL_MAX	25 000 000 cycles, roughly 620 ms
Decoupling cap	1 µF + 10 µF in parallel
Codebook tones	2000, 2400, 2900, 3500, 4500 Hz
Audio decoder window	±3 bin local maximum
Log record size	16 bytes
Settings size	32 bytes
Ring buffer slots	512
Retry attempts	5